

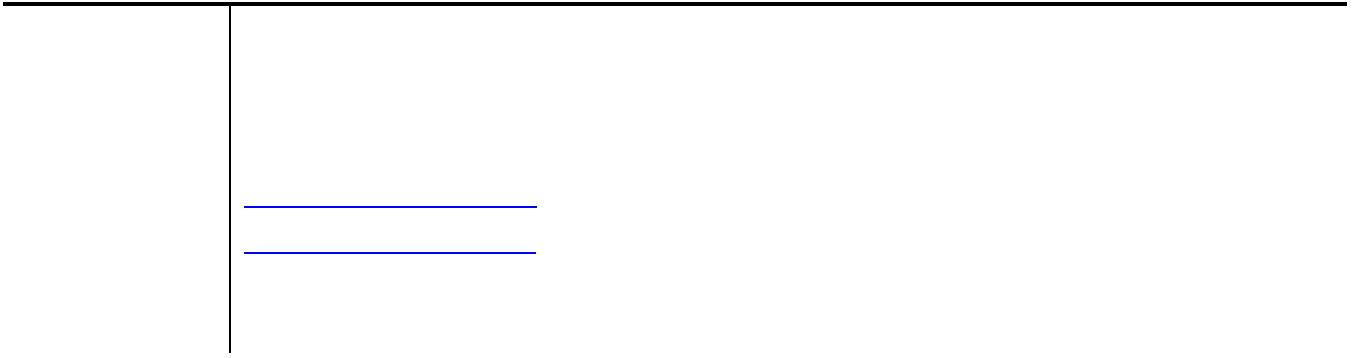
Autodesk® **Scaleform®**

Copyright Notice

Autodesk® Scaleform® 4.4

© 2014 Autodesk, Inc. All rights reserved.

Disclaimer



if

3.5.1	AS3_Global.h ' Obj\AS3_Obj_Global.xxx ‡ S V •	9
4.3.1	• X k V < ¥ z	20
4.3.2	_ 4 Ž 4 { V < ¥ z	21
4.3.3	„ u z x k z	23
4.3.4	_ 4 Ž 4 { ‡ [4] k	24
4.3.5	q u s V < ¥ z	24
6.1.1	FSCCommand - e 4 • , u a	29
6.1.2	ExternalInterface	31
6.2.1	ActionScript × / - • •	34
6.2.2	ActionScript g ^ • 4 s ¥ - ð •	37

1 . @ *

? :

? :

? :

2 { _ — “ ¥ z - ê



x
x
x

x
x

x

x

3 $\forall k \in \mathbb{Z}^+ \cdot \{ \dots \} - B \ddot{E} \acute{Y} \neg$

3.1 $\forall \neq k \geq 4$.

/

/

V

/

/

? :

3.2 y " - e ¥ f V •

\ \ \ \ \ \ \

\ \ \ \ \ \ \G

Æ Æ Æ Æ

3.3 Scaleform Player * J SWF — Ç - < ¥ s • 4 a

Æ Æ \ Æ \ Æ \

\ \ \ \ \ \

Ö 1:

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

? :

3.4 $g \neq \% \bullet \quad < 4 k - e \neq f \vee \bullet$

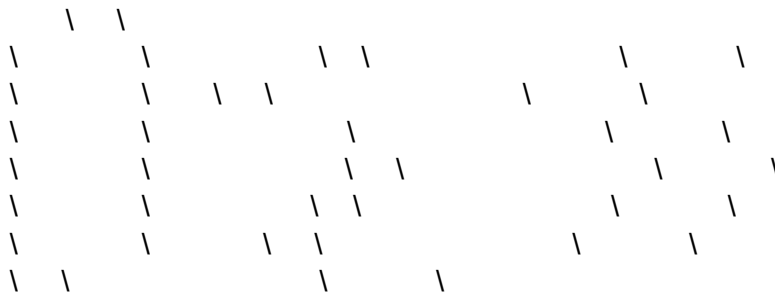
$\backslash \qquad \qquad \backslash \qquad \backslash$
 $\text{\AA} \qquad \qquad \text{\AA}$

$\tilde{O} \, 2$

3.5 Scaleform $\dots \bullet \{ - B \ddot{E} \acute{Y} \neg$

$\backslash \qquad \qquad \backslash \qquad \qquad \backslash$
 $\qquad \qquad \qquad \qquad \qquad \qquad \backslash$

\\



? :

\\

3.5.1AS3_Global.h ' Obj\AS3_Obj_Global.xxx ‡ S V •

ø Ý ¬ - î

4 d 4 ' Z ¥ j ¥ - ¼

4.1 Flash - ž ¥ r œ ¥ b

\

```
#include "Gfx_Kernel.h"
#include "Gfx.h"
#include "Gfx_Renderer_D3D9.h"
```

```
Loader          gfxLoader;
```

```
Ptr<MovieDef>    pUIMovieDef;
```

```
Ptr<Movie>       pUIMovie;
```

```
Ptr<Render::D3D9::HAL> pRenderHAL;
Ptr<Render::Renderer2D> pRenderer;
MovieDisplayHandle     hMovieDisplay;
```

```
//
Gfx::System          gfxInit;
```

```
gFx = new GFxTutorial();  
assert(gFx != NULL);  
if(!gFx->InitGFx())  
assert(0);
```

```
gfxLoader->SetLog(Ptr<Log>(*new Log()));
```

```
gFxLoader->SetFileOpener(pfileOpener);
```

```
pRenderHAL = *new Render::D3D9::HAL();  
if (!(pRenderer = *new Render::Renderer2D(pRenderHAL.GetPtr())))  
    return false;
```

```
pUIMovieDef = *gFxLoader.CreateMovie(UIMOVIE_FILENAME  
                                     Loader::LoadKeepBindData |  
                                     Loader::LoadWaitFrame1, 0);
```

```
pUIMovie = *pUIMovieDef->CreateInstance(true, 0, NULL);  
assert(pUIMovie.getPtr() != NULL);
```

```
pUIMovie->Advance(0.0f, 0, true);
```

```
MovieLastTime = timeGetTime();
```

î Ñ w

```
pUIMovie->SetBackgroundAlpha(0.0f);
```

```
pRenderHAL->InitHAL(  
    Render::D3D9::HALInitParams(pd3dDevice, presentParams,  
                                Render::D3D9::HALConfig_NoSceneCalls));
```

```
pRendererHAL->ShutdownHAL( );
```

```
delete gfx;  
gfx = NULL;
```

```
RECT windowRect = DXUTGetWindowClientRect();
DWORD windowWidth = windowRect.right - windowRect.left;
DWORD windowHeight = windowRect.bottom - windowRect.top;
pUIMovie->SetViewport(windowWidth, windowHeight, 0, 0,
                      windowHeight, windowHeight);
```

```
void AdvanceAndRender(void)
{
    DWORD mtime = timeGetTime();
    float deltaTime = ((float)(mtime - MovieLastTime)) / 1000.0f;
    MovieLastTime = mtime;

    pUIMovie->Advance(deltaTime, 0);
    pRenderer->BeginFrame();

    if (hMovieDisplay.NextCapture(pRenderer->GetContextNotify()))
```

```
    {  
        pRenderer->Display(hMovieDisplay);  
    }  
    pRenderer->EndFrame();  
}
```

```
g_pStateBlock->Capture();
```

```
gFx->AdvanceAndRender();
```

```
g_pStateBlock->Apply();
```

\

...

Ö 3

4.2 k c 4 œ ¥ b " 4 {

x
x

```
pUIMovie->SetViewScaleMode(Movie::SM_ShowAll);
```

```
pUIMovie->SetViewAlignment(Movie::Align_CenterRight);
```

4.3 Ě á V < ¥ z - ê ö

4.3.1 • X k V < ¥ z

```
void ProcessEvent(HWND hWnd, UINT uMsg, WPARAM wParam, LPARAM lParam,
                  bool *pbNoFurtherProcessing)
{
    int mx = LOWORD(lParam), my = HIWORD(lParam);
    if (pUIMovie)
    {
        if (uMsg == WM_MOUSEMOVE)
        {
            MouseEvent mevent(GFx::Event::MouseMove, 0, mx, my);
            pUIMovie->HandleEvent(mevent);
        }
        else if (pMovieButton && uMsg == WM_LBUTTONDOWN)
        {
            ::SetCapture(hWnd);
            MouseEvent mevent(GFx::Event::MouseDown, 0, mx, my);
            pUIMovie->HandleEvent(mevent);
        }
        else if (pMovieButton && uMsg == WM_LBUTTONUP)
        {
            ::ReleaseCapture();
            MouseEvent mevent(GFx::Event::MouseUp, 0, mx, my);
            pUIMovie->HandleEvent(mevent);
        }
    }
}
```

```
pMovie->SetViewport(screen_width, screen_height, 0, 0,
                    screen_width, screen_height, 0);
```

```
pMovie->SetViewport(screen_width, screen_height, 0, 0,
                    screen_width / 4, screen_height / 4, 0);
```

```
movie_width = screen_width / 6;
movie_height = screen_height / 6;
pMovie->SetViewport(screen_width, screen_height,
                    screen_width / 2 - movie_width / 2,
                    screen_height / 2 - movie_height / 2,
                    movie_width, movie_height);
```

4.3.2 4 4 { V < ¥ z

```
KeyEvent(EventType eventType = None,
          Key::Code code = Key::None,
          UByte asciiCode = 0,
          UInt32 wcharCode = 0,
          UInt8 keyboardIndex = 0)
```

```
CharEvent(UInt32 wcharCode, UInt8 keyboardIndex = 0)
```

y

f
f
f

y

'C'

y

y

```
void ProcessKeyEvent(Movie *pMovie, unsigned uMsg, WPARAM wParam,  
                    LPARAM lParam)
```

\

4.3.3 „u z x k z

```
bool processedMouseEvent = false;
if (uMsg == WM_MOUSEMOVE)
{
    MouseEvent mevent(GFx::Event::MouseMove, 0, (float)mx, (float)my);
    pUIMovie->HandleEvent(mevent);
    processedMouseEvent = true;
}
else if (uMsg == WM_LBUTTONDOWN)
{
    ::SetCapture(hWnd);
    MouseEvent mevent(GFx::Event::MouseDown, 0, (float)mx, (float)my);
    pUIMovie->HandleEvent(mevent);
    processedMouseEvent = true;
}
else if (uMsg == WM_LBUTTONUP)
{
    ::ReleaseCapture();
    MouseEvent mevent(GFx::Event::MouseUp, 0, (float)mx, (float)my);
    pUIMovie->HandleEvent(mevent);
}
```

```

        processedMouseEvent = true;
    }

    if (processedMouseEvent && pUIMovie->HitTest((float)mx, (float)my,
Movie::HitTest_Shapes))
        *pbNoFurtherProcessing = true;

```

4.3.4 `_4 Ž 4 { ‡ [ 4 ] k`

4.3.5 `q u s V < ¥ z`

Windows API Windows 7 WINVER=0x601

**RegisterTouchWindow
MsgProc WM_TOUCH**

```

if(uMsg == WM_TOUCH)
{
    ProcessTouchEvent(pUIMovie, uMsg, wParam, lParam);
}

```

ProcessTouchEvent WM_TOUCH

GFX::TouchEvent HandleEvent

SWF

GFx

MultitouchInterface

GFx::GestureEvent

Scaleform

SetMultitouchInterface

5 $\ddot{Y} 4] > V I ' \ddagger [\forall z * \# \%$

5.1 $\ddagger [\forall z - \hat{e} \ ' \ \check{s}$

$\backslash \quad \backslash \quad \backslash \quad \backslash$

おはようございます。
お元気ですか？
さみしかったです。

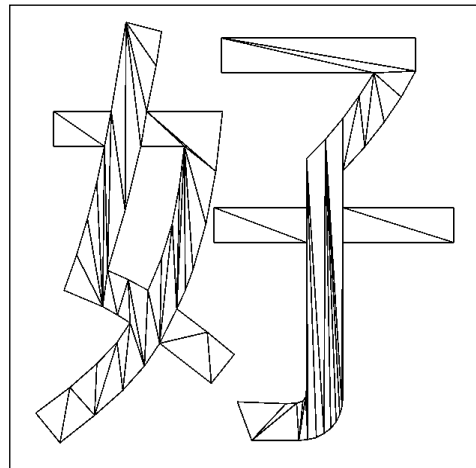


Ö 4a:

Ö 4b:



Ö 4c:



Ö 4d:

Õ 5

6 C++ Flash ActionScript - ü i • ÷

\

```
class OurFSCommandHandler : public FSCommandHandler
{
public:
    virtual void Callback(Movie* pmovie,
                          const char* pcommand, const char* parg)
    {
        GfxPrintf("FSCommand: %s, Args: %s", pcommand, parg);
    }
};
```

```
Ptr<FSCommandHandler> pcommandHandler = *new OurFSCommandHandler;
gFxLoader->SetFSCommandHandler(pcommandHandler);
```

FSCommand: ToggleFullscreen, Args:

is made by setting the ActionScript class for the “hudMgr” Symbol in d3d9guideAS3’s

```
if (strcmp(pcommand, "ToggleFullscreen") == 0)
    doToggleFullscreen = true;
```

```
if (doToggleFullscreen)
{
    doToggleFullscreen = false;
    DXUTToggleFullScreen();
}
```

6.1.2 External Interface

```
class OurExternalInterfaceHandler : public ExternalInterface
{
public:
    virtual void Callback(Movie* pmovieView,
```

```

        const char* methodName,
        const Value* args,
        unsigned argCount)
    {
        GFxPrintf("ExternalInterface: %s, %d args: ",
                    methodName, argCount);
        for(unsigned i = 0; i < argCount; i++)
        {
            switch(args[i].GetType())
            {
                case Value::VT_Null:
                    GFxPrintf("NULL");
                    break;
                case Value::VT_Boolean:
                    GFxPrintf("%s", args[i].GetBool() ? "true" : "false");
                    break;
                case Value::VT_Int:
                    GFxPrintf("%s", args[i].GetInt());
                    break;
                case Value::VT_Number:
                    GFxPrintf("%3.3f", args[i].GetNumber());
                    break;
                case Value::VT_String:
                    GFxPrintf("%s", args[i].GetString());
                    break;
                default:;
                    GFxPrintf("unknown");
                    break;
            }
            GFxPrintf("%s", (i == argCount - 1) ? "" : ", ");
        }
        GFxPrintf("\n");
    }
};

```

```

Ptr<ExternalInterface> pEIHandler = *new OurExternalInterfaceHandler;
gfxLoader.SetExternalInterface(pEIHandler);

```

```

function onSubmit(path:String) {
    ExternalInterface.call("MeshPath", path);
    CloseMeshPath();
}

```

```

function onFocusIn(ev: FocusHandlerEvent) {
    ExternalInterface.call("MeshPathFocus", true);
}

function onFocusOut(ev: FocusHandlerEvent) {
    ExternalInterface.call("MeshPathFocus", false);
}

```

```

Callback! MeshPathFocus, nargs = 1
    arg(0) = true

```

```

Callback! MeshPathFocus, nargs = 1
    arg(0) = false

```

```

if(strcmp(methodName, "MeshPathFocus") == 0 && argCount == 1 &&
    args[0].GetType() == Value::VT_Boolean) {
    if (args[0].GetType() == Value::VT_Boolean)
        gfx->SetTextboxFocus(args[0].GetBool());
}

```

```

if (uMsg == WM_SYSKEYDOWN || uMsg == WM_SYSKEYUP ||
    uMsg == WM_KEYDOWN || uMsg == WM_KEYUP ||
    uMsg == WM_CHAR)
{
    if (textboxHasFocus || wParam == 32 || wParam == 9)
    {
        ProcessKeyEvent(pUIMovie, uMsg, wParam, lParam);
        *pbNoFurtherProcessing = true;
    }
}

```

```

static bool doChangeMesh = false;
static wchar_t changeMeshFilename[MAX_PATH] = L"";

...

if(strcmp(methodName, "MeshPath") == 0 && argCount == 1)
{
    doChangeMesh = true;
    const char *filename = args[0].GetString();
    MultiByteToWideChar(CP_ACP, MB_PRECOMPOSED, filename, -1,
        changeMeshFilename, _countof(changeMeshFilename));
}

```

6.2 C++ H ActionScript 7

6.2.1 ActionScript × / - • •

```

void GfxTutorial::ProcessEvent(HWND hWnd, unsigned uMsg, WPARAM
    wParam, LPARAM lParam, bool *pbNoFurtherProcessing)
{

```

```

        int mx = LOWORD(lParam), my = HIWORD(lParam);

    if (pHUDMovie && uMsg == WM_KEYDOWN)
    {
        if (wParam == VK_F5)
        {
            int counter = (int)pHUDMovie
                ->GetVariableDouble("_root.counter");
            counter++;
            pHUDMovie->SetVariable("_root.counter",
                Value((double)counter));
            char str[256];
            sprintf_s(str, "testing! counter = %d", counter);
            pHUDMovie->SetVariable("_root.MessageText.text", str);
        }
    }

    ...

```

$f \nmid [4 \cdot \aleph_k - ?] !$:

) &

Ö 8

```
Value sceneData[3];
sceneData[0].SetString("Scene with shadow");
sceneData[1].SetString("Show shadow volume");
sceneData[2].SetString("Shadow volume complexity");
pUIMovie->SetVariableArraySize("_root.SceneData", 3);
pUIMovie->SetVariableArray(Movie::SA_Value,
                           "_root.SceneData", 0, sceneData, 3);
```

\

? :

6.2.2 ActionScript g ^ • 4 s ¥ - ð .

```
...

else if (wParam == VK_F6)
{
    bool retval = pHUDMovie->Invoke("root.ui.hud.OpenMeshPath", "");
    GfxPrintf("root.ui.hud.OpenMeshPath returns '%d'\n", (int) retval);
}
else if (wParam == VK_F7)
{
    const char *retval = pHUDMovie->Invoke(
        "root.ui.hud.CloseMeshPath", "");
    GfxPrintf("root.ui.hud.CloseMeshPath returns '%d'\n", (int) retval);
}

...
```

6.3 3 / - Flash ‡ S V • Ñ - â &

```
var mclLoader:MovieClipLoader = new MovieClipLoader();
```

```
function loadMovie(url:String):void {  
    var loader = new Loader();  
    loader.load( new URLRequest( url ) );  
    addChild( loader )  
}
```

loadMovie

loadMovie

loadMovie

```
function loadMovie(url:String):void {
    var loader = new Loader();
    loader.load( new URLRequest( url ) );
    loader.contentLoaderInfo.addEventListener( Event.OPEN, handleLoadOpen,
                                                false, 0, true );
    loader.contentLoaderInfo.addEventListener( Event.INIT, handleLoadInit,
                                                false, 0, true );
    loader.contentLoaderInfo.addEventListener( Event.PROGRESS,
                                                handleLoadProgress, false, 0, true );
    loader.contentLoaderInfo.addEventListener( Event.COMPLETE,
                                                handleLoadComplete, false, 0, true );
    addChild( loader )
}

function handleLoadOpen( e:Event ):void {
    trace("Event.OPEN - Load Started!");
}

function handleLoadInit( e:Event ):void {
    trace("Event.INIT - Loaded Content Ready!");
}
```

```
function handleLoadProgress( e:Event ):void {  
    trace("Event.PROGRESS - Load Progress!");  
}  
function handleLoadComplete( e:Event ):void {  
    trace("Event.COMPLETE - Load Complete!");  
}
```

7 x a k s • 4 7 - ž ¥ r œ ¥ b

```

Material {
    1.000000;1.000000;1.000000;1.000000;;
    40.000000;
    1.000000;1.000000;1.000000;;
    0.000000;0.000000;0.000000;;
    TextureFilename {
"cellwallRT.jpg";
    }
}

```

```

pd3dDevice->CreateTexture(rtw,rth,0,
    D3DUSAGE_RENDERTARGET|D3DUSAGE_AUTOGENMIPMAP, D3DFMT_A8R8G8B8,
    D3DPPOOL_DEFAULT, &g_pRenderTex, 0);

```

```

g_Background[0].m_pTextures[BACK_WALL] = g_pRenderTex;
ptex->Release();

```

a.

```

pd3dDevice->GetRenderTarget(0, &poldSurface);
pd3dDevice->GetDepthStencilSurface(&poldDepthSurface);

```

b.

```

ptex->GetSurfaceLevel(0, &psurface);
HRESULT hr = pd3dDevice->SetRenderTarget(0, psurface );

```

c.

8 OpenGL g ¥ % •

9 GFxExport Q ž " Đ ê ö

```
gfxexport -i DDS -c d3d9guideAS3.swf
gfxexport -i DDS -c fxplayer.swf
```

```
\      \      \
\
```

\

10 $f^{-0.7}$

